

Frontend Integration Guide v3: Scoped Drill-Down & Autocomplete Flow

Hello Frontend Team! 🙌

We have completely upgraded the Analytics Backend to support the new Parameter Screen (Date, Report, Code, Scope) and Role-Based Access Control (RBAC).

The backend now dynamically handles hierarchies based on the user's assigned roles. **You do not need to write any if/else logic for user roles.** The database dictates the flow.

The 4-Step UI Flow

1. **Bootstrap:** Fetch permitted Reports and Scopes on component mount.
2. **Autocomplete:** As the user types in the "Code" field, fetch suggestions from the backend.
3. **Initial Fetch:** Send the first drill-down request using the UI filters and the starting dimension of the selected Scope.
4. **Drill Down:** Append row names to the filters array and fetch the next dimension.

Step 1: Bootstrapping the Dropdowns

On page load, call this API. It will automatically read the user's JWT token and only return the Scopes (Views) they are allowed to see.

- **Endpoint:** GET /api/v1/analytics/config
- **Headers:** Authorization: Bearer <JWT_TOKEN>

Response Example (For a User who only has Bank & Branch access):

```
[
  {
    "reportCode": "PNL",
    "displayName": "PNL Report",
    "views": [
      {
        "viewCode": "PNL_BANK",
        "displayName": "Bank Level",
        "hierarchy": ["Geography", "Circle", "Cgl", "Product",
"Branch"],
        "availableMetrics": [{"logicalName": "TotalAmount",
"displayName": "Amount"}]
      },
      {
        "viewCode": "PNL_BRANCH",
        "displayName": "Branch Level",
        "hierarchy": ["Cgl", "Product"],
        "availableMetrics": [{"logicalName": "TotalAmount",
"displayName": "Amount"}]
      }
    ]
  }
]
```

```
]
}
```

UI Action: Use this JSON to populate the "Report" and "Scope" dropdowns. Store the viewCode (e.g., PNL_BANK) and the hierarchy array in your React state.

Step 2: The Autocomplete "Code" API

When the user selects a report (e.g., YSA) and starts typing a Head Code (e.g., "100"), call this API to get suggestions.

- **Endpoint:** GET /api/v1/analytics/heads/search?reportName=ysa_report&search=100
- **Headers:** Authorization: Bearer <JWT_TOKEN>

Response Example:

```
[
  {
    "headCode": "1001",
    "headName": "Bank Charges"
  },
  {
    "headCode": "1002",
    "headName": "Interest on Loans"
  }
]
```

UI Action: Render a dropdown of suggestions. When the user selects one, extract the headCode string (e.g., "1001"). You will pass this in the filters array later.

Step 3: Mapping UI Inputs to Backend Filters

When the user clicks "**Fetch Data**", you must map their screen inputs to the exact logicalField names the backend expects.

UI Input Field	Logical Field Name (logicalField)	Example Value
Date	ReportDate	"2026-03-31"
Report (Hidden/Derived)	ReportType	"pnl_report" / "ysa_report"
Head Code (From Step 2)	HeadCode	"1001"
If Scope = Circle Level	Circle	"Kolkata"
If Scope = Branch Level	Branch	"01111"

Step 4: The Initial Data Fetch (Based on Scope)

- **Endpoint:** POST /api/v1/analytics/drill-down
- **Headers:** Authorization: Bearer <JWT_TOKEN>, Content-Type: application/json

You must send the viewCode, the **first item** in the hierarchy array for that view, and the accumulated filters.

Scenario A: Bank Level Scope Selected

Because the Bank hierarchy is ["Geography", "Circle", "Cgl", ...], you ask for Geography.

```
{
  "viewCode": "PNL_BANK",
  "dimensions": ["Geography"],
  "metrics": ["TotalAmount"],
  "filters": [
    { "logicalField": "ReportDate", "operator": "EQUALS", "value":
"2026-03-31" },
    { "logicalField": "ReportType", "operator": "EQUALS", "value":
"pnl_report" },
    { "logicalField": "HeadCode", "operator": "EQUALS", "value":
"1001" }
  ]
}
```

Scenario B: Circle Level Scope Selected (User typed "Kolkata")

Because the Circle hierarchy is ["Cgl", "Product", "Branch"], you ask for Cgl. Notice how Circle is now passed as a filter!

```
{
  "viewCode": "PNL_CIRCLE",
  "dimensions": ["Cgl"],
  "metrics": ["TotalAmount"],
  "filters": [
    { "logicalField": "ReportDate", "operator": "EQUALS", "value":
"2026-03-31" },
    { "logicalField": "ReportType", "operator": "EQUALS", "value":
"pnl_report" },
    { "logicalField": "HeadCode", "operator": "EQUALS", "value":
"1001" },
    { "logicalField": "Circle", "operator": "EQUALS", "value":
"Kolkata" }
  ]
}
```

Scenario C: Branch Level Scope Selected (User typed "01111")

Because the Branch hierarchy is ["Cgl", "Product"], you ask for Cgl. Notice how Branch is passed as a filter.

```
{
  "viewCode": "PNL_BRANCH",
  "dimensions": ["Cgl"],
  "metrics": ["TotalAmount"],
  "filters": [
```

```

    { "logicalField": "ReportDate", "operator": "EQUALS", "value":
"2026-03-31" },
    { "logicalField": "ReportType", "operator": "EQUALS", "value":
"pnl_report" },
    { "logicalField": "HeadCode", "operator": "EQUALS", "value":
"1001" },
    { "logicalField": "Branch", "operator": "EQUALS", "value": "01111"
}
]
}

```

Step 5: The Response & Subsequent Drills

The backend will respond with the aggregated data, generated IDs, and a hasChildren boolean.

Example Response:

```

{
  "meta": {
    "status": "SUCCESS",
    "level": "Cgl",
    "rowCount": 2,
    "executionTimeMs": 120
  },
  "data": [
    {
      "id": "cgl_1260505001",
      "name": "1260505001",
      "TotalAmount": 4057156.00,
      "hasChildren": true
    }
  ]
}

```

To Drill Down Further: When the user clicks [+] on CGL 1260505001:

1. Take the current row's name (1260505001).
2. Add it to your filters array (logicalField: "Cgl").
3. Grab the *next* item from the hierarchy array (e.g., "Product").
4. Send the POST request again.

Note: If your frontend utilizes a global Encryption Interceptor (like AES-256 for all POST bodies), ensure the backend is configured to decrypt the payload, or bypass the interceptor for this specific endpoint to send raw JSON.